Building a RAG System for Customer Support - L1

October 2025

Problem Statement
Customer Support System (Level 1 Coverage - L1)

Repetitive Query Handling

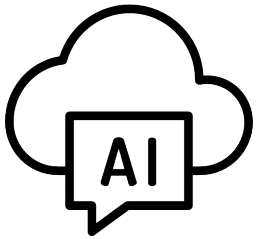
Support teams face many repeated customer queries, which slow down response times and increase workload.

Inconsistent Responses

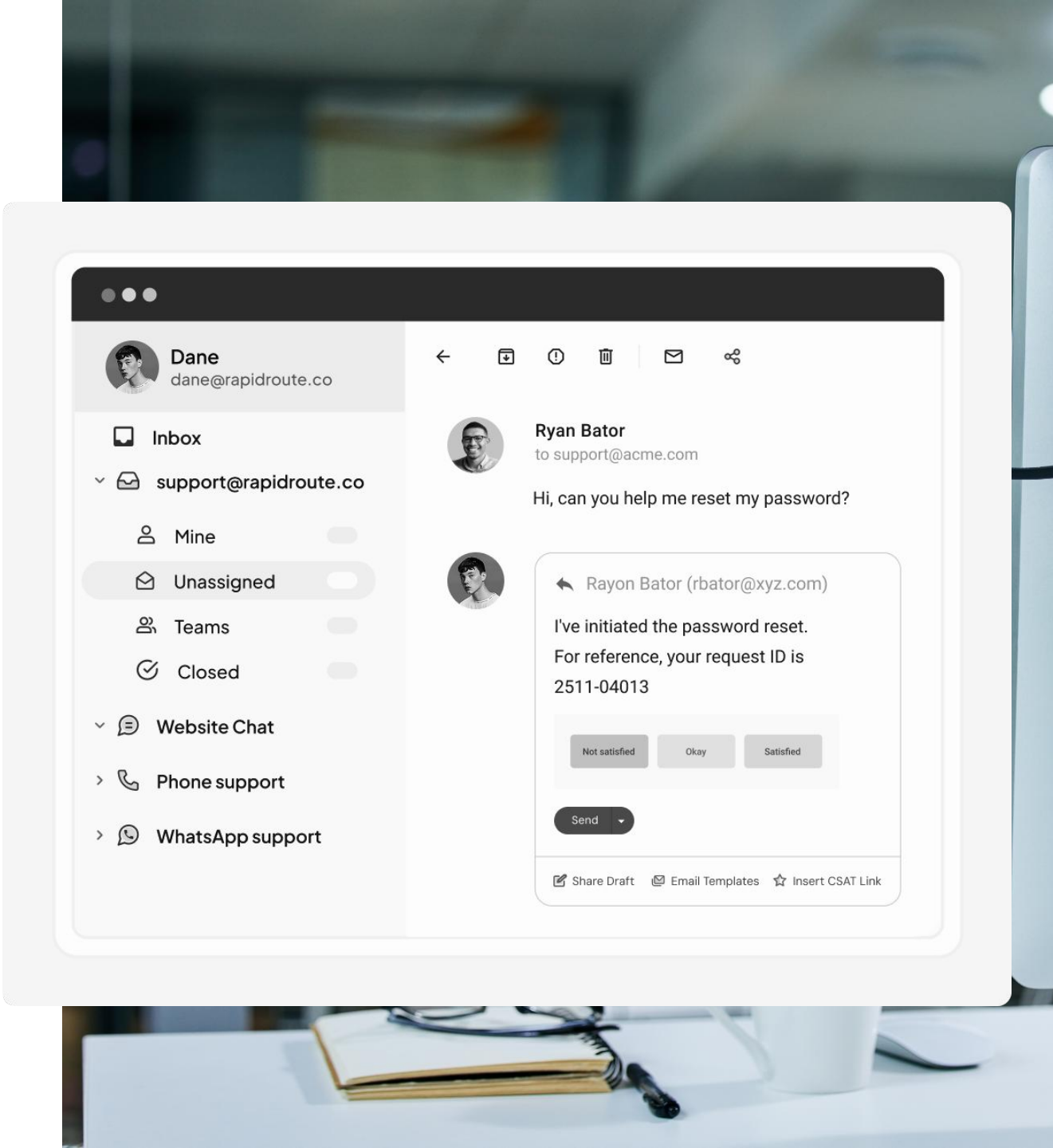
Manual answering leads to inconsistent information, which can negatively impact customer satisfaction and trust in support teams.

Need for Intelligent Systems

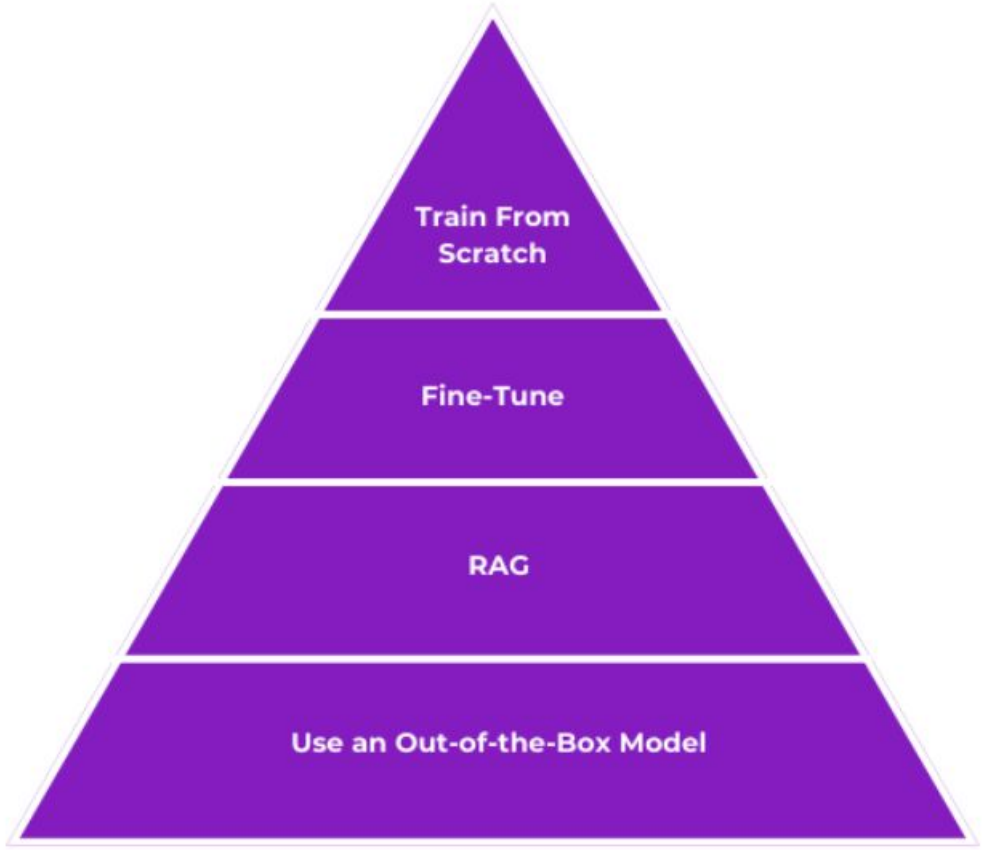
Accurate, intelligent information retrieval systems are needed to streamline support and ensure reliable, consistent communication.



AI Agent



LLM Design Hierarchy



Training From Scratch

Building an LLM from the ground up is rare due to vast data needs and intensive tuning, making it resource-heavy.

Domain Fine-Tuning

Fine-tuning adapts existing models with domain-specific data, lowering costs but relying on high-quality examples.

Retrieval-Augmented Generation

RAG combines existing models with external knowledge sources to enhance responses and flexibility.

Out-of-the-Box Solutions

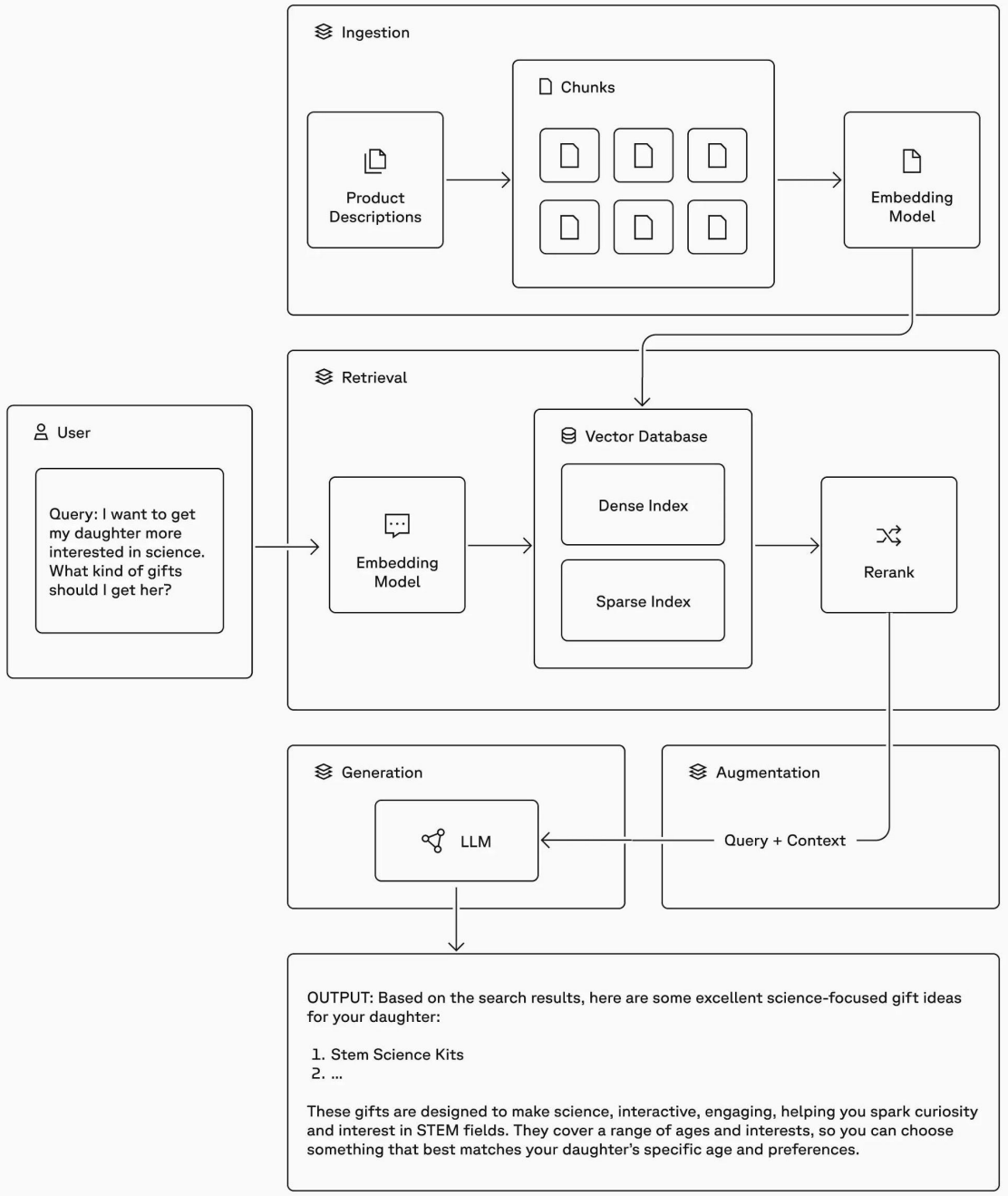
Pre-built models offer rapid deployment and ease of use but are limited in customization.

What is RAG?

Retrieval-Augmented Generation (RAG) combines:

- **Ingestion:** Load and preprocess `documents` into the vector databse for indexing
- **Retrieval:** Fetches relevant documents from a vector database.
- **Generation:** Uses LLMs to create accurate, context-aware answers.

Ensures answers are grounded in **trusted internal data**.



What is RAG?

Combining Retrieval and Generation

RAG merges document retrieval and text generation, allowing language models (LLMs) to access and use relevant information on demand.

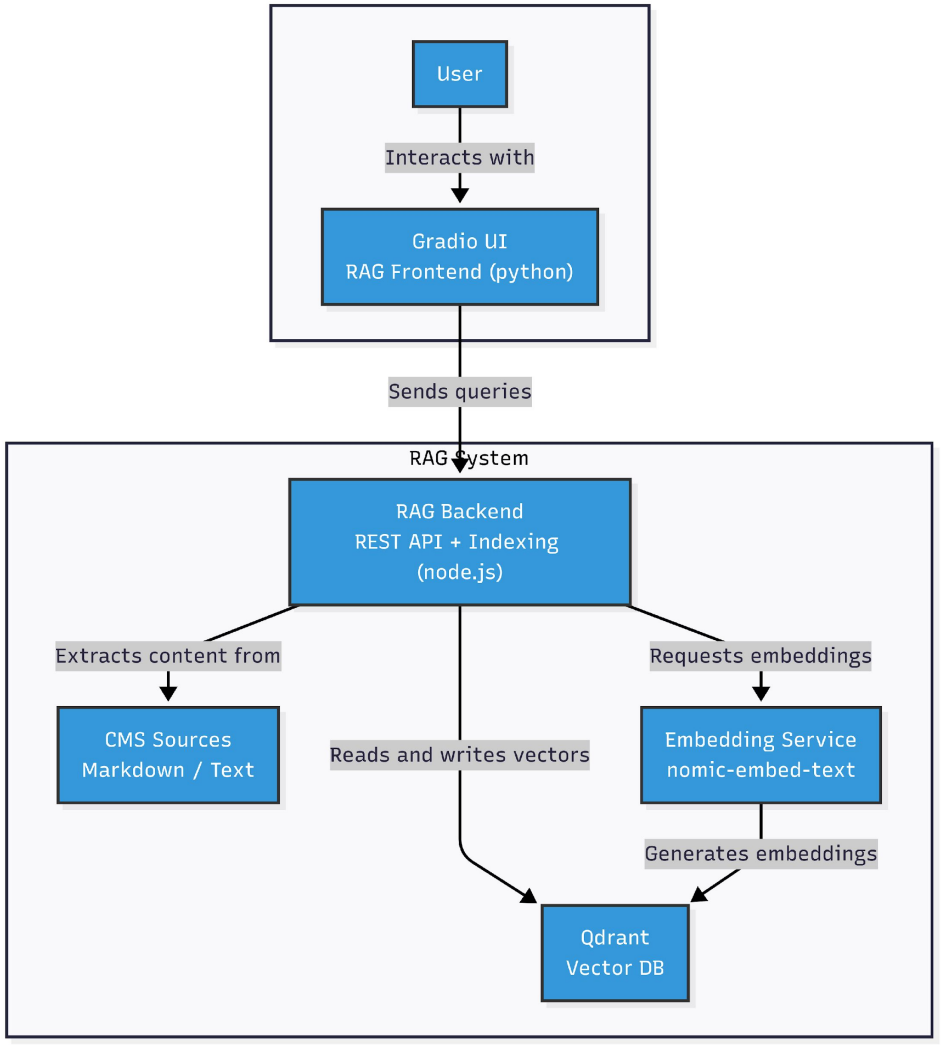
Context from Vector Databases

RAG fetches relevant documents from a vector database, providing valuable context for generating precise and informed answers.

Grounded and Trustworthy Output

By grounding answers in trusted internal data, RAG increases the reliability and trustworthiness of language model outputs.

Architecture Overview



Data Collection and Indexing

Content is extracted from local .md and .txt CMS sources, then processed by a custom Node.js indexer using **LlamaIndex**.

Vector Database and Embeddings

Processed data is stored in **Qdrant**, a vector database, using embeddings generated by nomic-embed-text technology.

API Orchestration and User Interface

The backend is handled by a Node.js REST API, while Gradio powers the interactive frontend for users.

Architecture Overview
LLMs models

- **Data Source:** Local `.md` & `.txt` files (CMS extracts)
- **Indexing:** Custom Node.js indexer + LlamaIndex
- **Vector Database:** Qdrant

Ollama is a **local runtime for large language models (LLMs)** 🧠 that makes it easy to **download, run, and manage open-source LLMs** directly on your machine — without needing cloud APIs.

- 🔒 **Privacy & control** — since everything runs locally, no data leaves your system.

- **Models (Ollama):**
  - **Embeddings**: `nomic-embed-text (274MB)`
  - **Default** LLM: `llama3.2`
  - **Supported**:
    - llama3.1 (8b), llama3.2 (3b),
    - gpt-oss (20b),
    - gemma3 (4b, 12b),

- **Backend:** Node.js server (REST API + orchestration)
- **Frontend/UI:** Gradio

Local Model Execution with Ollama

All Models Run Locally

Running models locally removes API dependencies, providing users with greater privacy and control over data processing.

Flexible Model Support

Ollama supports multiple LLM families, such as Llama, Gemma, GPT-OSS, enhancing model flexibility.
GPT-5 variants used via the OpenAI platform.

Enhanced Privacy and Performance

Local execution delivers improved privacy, cost efficiency, and faster response times compared to cloud-based solutions.

Architecture Overview
Vector Database Choices

Pinecone

Milvus

Weaviate

Qdrant

Chroma: Developer-Friendly Simplicity
Chroma is a lightweight, developer-focused vector database designed for rapid prototyping and smaller-scale applications, with a simple API and quick setup.

Pinecone: Managed Scalability
Pinecone is a managed vector database offering high scalability and performance, but it comes with a higher operational cost.

Weaviate: Feature-Rich and Modular
Weaviate provides a flexible and modular platform with rich features suited for various vector search needs.

Milvus: Enterprise Clustering
Milvus supports enterprise-grade clustering, making it suitable for large-scale deployments and robust performance requirements.

Qdrant: Efficient Open-Source
Qdrant is an open-source, efficient vector database ideal for self-hosted environments and the preferred option for our needs.

Architecture Overview
Why Choose Qdrant?

Open Source and Cost-Effective

Qdrant provides an accessible solution for vector similarity search with no licensing costs, making it budget-friendly for all users.

High Performance Search

It enables rapid and accurate searches even on massive datasets, ensuring excellent performance for demanding applications.

Easy Integration and Scalability

Qdrant integrates seamlessly with popular languages like Node.js and Python, and scales flexibly to support growing data and user needs.

Backend Architecture

Core of the system: Modular document search with **Elasticsearch + Qdrant**

Indexes `.md` & `.txt` documents for search and RAG

Provides a RESTful API to support search and RAG workflows

Key Features:

- Exact & fuzzy search via **Elasticsearch**
- Vector similarity search & RAG responses via **Qdrant**
- REST API endpoints for search, update, and response generation
- Optional Gradio UI for interactive access

Frontend & UI

Customizable Input Options

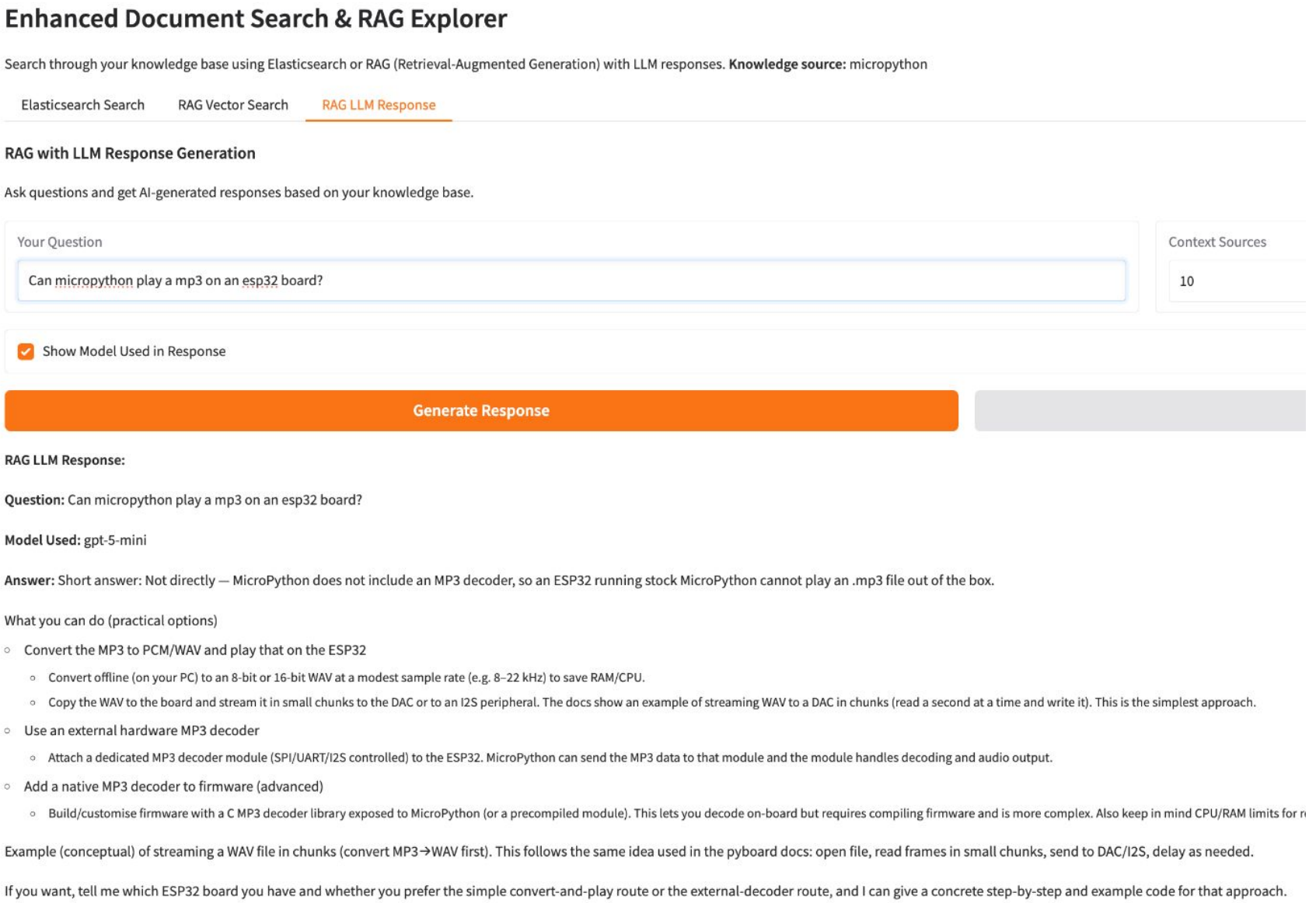
Users can input questions and choose context sources, enabling tailored and flexible interactions for various needs.

LLM Model Selection

A dropdown lets users select different language models, clearly displaying which model generated each response.

Transparent Source Listings

The UI shows source information with relevance scores, helping users understand and trust the provided answers.



Key Solution Benefits

- **Improved Accuracy:**
  Responses are grounded in `.md` and `.txt` files extracted from the CMS, ensuring that answers are consistent and based on verified information.

- **Faster Response Times:**
  By retrieving relevant context and running models locally, the system provides instant answers, reducing wait times for both support agents and end-users.

- **Operational Efficiency:**
  Automates repetitive queries and reduces manual effort, allowing support teams to focus on more complex issues instead of answering FAQs.

- **Privacy and Cost Control:**
  Running models locally with Ollama means no data leaves the environment and external API costs are avoided. This setup ensures compliance with data policies and predictable costs.

- **Scalability and Flexibility:**
  The modular design supports adding new data sources, expanding the knowledge base, and experimenting with multiple LLMs, making it future-proof and adaptable.

Unlocking Future Potential

Content / CMS Automated Content Synchronization

Daily sync ensures .md or .txt files are always up-to-date, enhancing content reliability and workflow efficiency.

Multi-Language Accessibility

Supporting multiple languages increases accessibility, allowing users worldwide to enjoy an improved experience.

Tailored Insights and Interactivity

Specific models deliver precise insights; chatbot and analytics dashboard support interactive, data-driven decision-making.

- **Chatbot Integration:** Enables end-users or support agents to interact with the knowledge base in real time, asking follow-up questions and drilling deeper into topics.
- **Analytics Dashboard:** Provides visibility into query trends, frequent issues, and system performance. This makes the support workflow **data-driven**, helping teams continuously improve processes and anticipate customer needs.

Thank you for your presence.

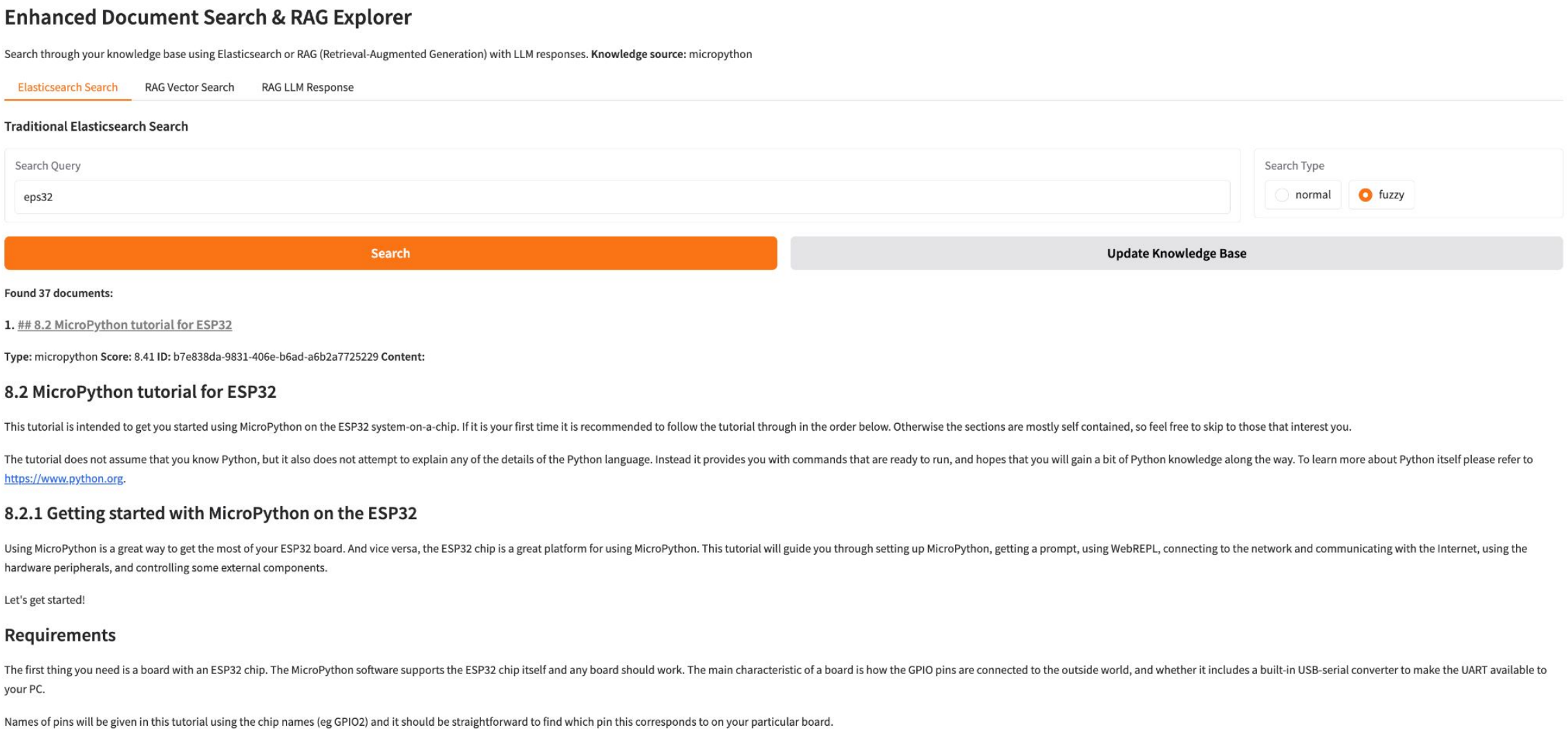
Bogdan Mustata
Software Architect | Cloud & AI

bmustata@yahoo.com

Thank you

GitHub Repo
https://github.com/bmustata/rag_exploration

Extra


Enhanced Document Search & RAG Explorer
Search through your knowledge base using Elasticsearch or RAG (Retrieval-Augmented Generation) with LLM responses. Knowledge source: micropython
Elasticsearch Search
RAG Vector Search
RAG LLM Response
Traditional Elasticsearch Search
Search Query
eps32
Search Type
normal
fuzzy
Search
Update Knowledge Base
Found 37 documents:
1. ## 8.2 MicroPython tutorial for ESP32
Type: micropython Score: 8.41 ID: b7e838da-9831-406e-b6ad-a6b2a7725229 Content:
8.2 MicroPython tutorial for ESP32
This tutorial is intended to get you started using MicroPython on the ESP32 system-on-a-chip. If it is your first time it is recommended to follow the tutorial through in the order below. Otherwise the sections are mostly self contained, so feel free to skip to those that interest you.
The tutorial does not assume that you know Python, but it also does not attempt to explain any of the details of the Python language. Instead it provides you with commands that are ready to run, and hopes that you will gain a bit of Python knowledge along the way. To learn more about Python itself please refer to https://www.python.org.
8.2.1 Getting started with MicroPython on the ESP32
Using MicroPython is a great way to get the most of your ESP32 board. And vice versa, the ESP32 chip is a great platform for using MicroPython. This tutorial will guide you through setting up MicroPython, getting a prompt, using WebREPL, connecting to the network and communicating with the Internet, using the hardware peripherals, and controlling some external components.
Let's get started!
Requirements
The first thing you need is a board with an ESP32 chip. The MicroPython software supports the ESP32 chip itself and any board should work. The main characteristic of a board is how the GPIO pins are connected to the outside world, and whether it includes a built-in USB-serial converter to make the UART available to your PC.
Names of pins will be given in this tutorial using the chip names (eg GPIO2) and it should be straightforward to find which pin this corresponds to on your particular board.

Extra

Enhanced Document Search & RAG Explorer

Search through your knowledge base using Elasticsearch or RAG (Retrieval-Augmented Generation) with LLM responses. **Knowledge source:** micropython

Elasticsearch Search | RAG Vector Search | RAG LLM Response

RAG Vector Similarity Search

Search using vector embeddings to find semantically similar content.

Search Query: eps32

Top K Results: 10

RAG Search | Update RAG Knowledge Base

RAG Vector Search - Found 10 documents:

1. The ESP32 port also supports the machine.ADC API:

Score: 0.3755 **Type:** micropython **Filename:** micropython-docs8-13.md **Content:**

```
ADC. atten ( atten ) Equivalent to ADC.init(atten=atten) . ADC. width ( bits ) Equivalent to ADC.block().init(bits=bits) .
```

The only chip that can switch resolution to a lower one is the normal esp32. The C2 & S3 are stuck at 12 bits, while the S2 is at 13 bits.

For compatibility, the ADC object also provides constants matching the supported ADC resolutions, per chip:

ESP32:

- ADC.WIDTH_9BIT = 9 · ADC.WIDTH_10BIT = 10 · ADC.WIDTH_11BIT = 11 · ADC.WIDTH_12BIT = 12 ESP32 C3 & S3: · ADC.WIDTH_12BIT = 12 ESP32 S2: · ADC.WIDTH_13BIT = 13 ADC. deinit ()

Provided to deinit the adc driver.

8.13 Pulse Counter (pin pulse/edge counting)

The ESP32 provides up to 8 pulse counter peripherals depending on the hardware, with id 0..7. These can be configured to count rising and/or falling edges on any input pin.

Use the esp32.PCNT class:

Extra

Enhanced Document Search & RAG Explorer

Search through your knowledge base using Elasticsearch or RAG (Retrieval-Augmented Generation) with LLM responses. **Knowledge source:** micropython

Elasticsearch Search | RAG Vector Search | RAG LLM Response

RAG with LLM Response Generation

Ask questions and get AI-generated responses based on your knowledge base.

Your Question: Can micropython play a mp3 on an esp32 board?

Context Sources: 10

LLM Model: online gpt-5-mini

Show Model Used in Response

Generate Response | Update RAG Knowledge Base

RAG LLM Response:

Question: Can micropython play a mp3 on an esp32 board?

Model Used: gpt-5-mini

Answer: Short answer: Not directly — MicroPython does not include an MP3 decoder, so an ESP32 running stock MicroPython cannot play an .mp3 file out of the box.

What you can do (practical options)

- Convert the MP3 to PCM/WAV and play that on the ESP32
  - Convert offline (on your PC) to an 8-bit or 16-bit WAV at a modest sample rate (e.g. 8–22 kHz) to save RAM/CPU.
  - Copy the WAV to the board and stream it in small chunks to the DAC or to an I2S peripheral. The docs show an example of streaming WAV to a DAC in chunks (read a second at a time and write it). This is the simplest approach.
- Use an external hardware MP3 decoder
  - Attach a dedicated MP3 decoder module (SPI/UART/I2S controlled) to the ESP32. MicroPython can send the MP3 data to that module and the module handles decoding and audio output.
- Add a native MP3 decoder to firmware (advanced)
  - Build/customise firmware with a C MP3 decoder library exposed to MicroPython (or a precompiled module). This lets you decode on-board but requires compiling firmware and is more complex. Also keep in mind CPU/RAM limits for realtime decoding.

Example (conceptual) of streaming a WAV file in chunks (convert MP3→WAV first). This follows the same idea used in the pyboard docs: open file, read frames in small chunks, send to DAC/I2S, delay as needed.

If you want, tell me which ESP32 board you have and whether you prefer the simple convert-and-play route or the external-decoder route, and I can give a concrete step-by-step and example code for that approach.